프로필

이름	안태민
생년월일	2001.02.12
번호	010.5513.9724
메일	robotaeminan@gmail.com
주소	경기도 안산시

학력

경기모바일과학고등학교 졸업 [2017.03~2020.01]
인하공업전문대학 [2020.03~재학 중]

- 컴퓨터정보학과 3 학년
- 학점 : 3.35 / 4.5

경력

지뉴소프트 인턴 [2025.07~2025.08]

지뉴소프트 현장실습 기간 프로젝트

프로젝트명 : 조선일보 AI POC

팀원 : 3 명(RAG, AWS 인프라 구성 1 명, AI 채팅 Back-End 및 AWS EC2 배포 2 명)

기간 : [2025.07.21~2025.08.06]

목적 : 사용자가 질의를 통해 조선일보 뉴스 데이터 내에서 RAG 를 통해 해당 질의에 해당하는 검색 결과 정리 후 종합 설명한 답변 제공하는 기능을 실현할 수 있는지 검증하기 위한 프로젝트

사용 기술

Front-End	Python(Streamlit)
Back-End	Python(Flask), Java(Spring Boot)
Database	PostgreSQL, FAISS
협업 및 버전관리	Git, GitLab
외부 서비스	Upstage API(Embeddings, Chat), AWS(OpenSearch, EC2, Lambda)

Python Stremlit 라이브러리를 사용한 채팅 테스트 화면

흐름 구성

- 사용자가 조선일보 AI를 통해 뉴스 검색을 위한 질문 입력
- Spring Back-End가 요청을 받아 LLM을 통해 키워드 및 검색 시간 추출
- Flask BacK-End에서 뉴스 조회 요청을 받아 키워드 및 검색 시간을 이용하여 Opensearch에서 뉴스 데이터 조회
  Opensearch 내재 VectorDB를 이용하여 조회한 뉴스에 대한 리랭킹 진행
- Spring Back-end가 리랭킹 된 뉴스와 사용자의 질문을 LLM에 요청하여 최종 답변을 받아 사용자에게 제공

담당 역할

- 해당 POC 프로젝트의 서포트(Json 데이터 형식 추가) 및 기능 테스트 담당
- AI Chat 기능을 제작한 Spring Boot Back-End AWS EC2 배포
- 최종 답변에 날짜 범위 키워드에 따른 검색 기간 범위 반환을 위한 LLM 요청 프롬프트 조정

문제 및 해결 과정

- **Lambda와 Spring Boot 호환 이슈**들로 인해 EC2로 변경
- **API Gateway를 거쳐서 Json 요청 데이터가 Spring Boot API로 들어갈 때 ? 로 깨지는 문제** 발생,
  AWS API Gateway 콘솔 설정 **이진 미디어 유형을 지정하여 해결**
- AWS Lambda는 AI 채팅 기능에서의 **스트리밍 기능을 제한적으로 지원**해 주기에 **콜드 스타트로 인한 시간까지 고려하여 EC2로 변경하여 배포**

이전 버전에서는 엔드포인트 끝에 슬래시가 붙어있어도 처리해 줬던 반면 Spring Boot 3으로 넘어가면서 v1/ chat, v1/chat/ 두 개의 경로를 다른 경로로 컨트롤러가 v1/chat/ 이 경로로 요청을 받는다고 하면 슬래시가 삭제되어 넘어가기에
요청 엔드포인트가 API와 매치되지 않아 컨트롤러 코드를 변경

```
@RequestMapping("/api/v1/chat")
@Tag(name = "채팅방", description = "채팅방 관리 API")
@Description("채팅 방 컨트롤러")
public class ChatRoomsController {

    private final ChatRoomsService chatRoomsService;

    @Operation(  taemin +1
            summary = "채팅방 생성",
            description = "채팅방을 생성합니다."
    )
    @ApiResponses(value = {
            @ApiResponse(responseCode = "200", description = "방 생성 성공"),
            @ApiResponse(responseCode = "400", description = "잘못된 요청"),
    })

    @PostMapping({"", "/"})
    public ResponseEntity<BaseResponse> createChatRooms(@RequestBody @Valid CreateChatRoomsRequest createChatRoomsRequest) {
        CreateChatRoomsResponse response = chatRoomsService.createChatRooms(createChatRoomsRequest);
        return ResponseEntity.ok(BaseResponse.of(response));
    }
```

해결 후 컨트롤러 코드

느낀 점

- AWS 서비스를 사용하는 데에 있어 지식이 부족하여 프로젝트에서 사용하려면
  학습 및 실습으로 익숙해지는 시간이 필요하다고 느꼈습니다.
  현재 3학년 2학기 클라우드 컴퓨팅 강의를 통해 AWS를 학습 중입니다.
- 개발에 직접 참여하지 않아 코드를 작성하는 시간보다 읽는 시간이 더 길었으며, 코드를 잘 읽는 전략이 필요하다고 느꼈습니다. 언어 및 익숙하지 않은 프레임워크라고 하더라도 결국 흐름이 존재하기 때문에 그 흐름을 따라 코드를 읽다 보면 조금이나마 더 빠르게 이해할 수 있다고 생각합니다.
- 협업을 위해 다양한 컨벤션들이 존재합니다. Commit, Issue, code 등 효율적으로 협업을 하기 위해서는 많은 컨벤션들은 아니더라도 Issue, PR, Commit 등에 대해서는 필요하다고 생각합니다. 현장실습을 진행하며 Issue, PR 생성에는 작성 템플릿이 있으면 작성하기 수월하며, 보는 사람도 편하다고 생각합니다.
- 회사에서의 POC를 참여하며 첫 실무를 경험했습니다. 개발 능력도 중요하지만, 소통을 제대로 하지 않으면 혼선이 생기고, 충돌이 생기기 쉬우며, 실제로 조선일보 AI POC 프로젝트를 진행하며 채팅 기능까지 RAG 구성하는 팀원이 하는 것으로 알고 있어 서로 다른 API에서 기능이 겹쳤습니다.
  효율적인 소통을 위해서는 문서도 중요하며, 문서 보는 것을 습관화 하는 것도 중요하다고 생각합니다.

프로젝트명 : RAG 성능 향상

개인 프로젝트로 진행

기간 : [2025.07.14~2025.07.18]

목적 : RAG 기초 실습 및 당시 진행중이었던 제공 문서에 대해 RAG 성능 향상

GitHub : https://github.com/taemin01/python-rag-pipeline

사용 기술

Language	Python
Database	Chroma DB(Home Server DB)
버전관리	Git, GitHub
외부 서비스	HuggingFace(Embedding Model), Upstage(Solar Pro LLM API, Document Parse)

흐름 구성

- 사내 Document Parse를 통해 문서 데이터를 Json으로 받아 Python으로 로드
- LangChain RecursiveCharacterTextSplitter를 사용하여 텍스트 청킹 진행
- 페이지 정보 메타데이터를 보존하며 텍스트를 HuggingFace의 sentence-transformers/all-MiniLM-L6-v2 한국어 처리에 특화된 모델을 사용하여 임베딩 진행 후 Chroma DB에 적재
- 이후 사용자의 질문을 받으면 텍스트를 임베딩하여 Chroma DB 쿼리를 통해 유사도 검색을 하여 결과를 반환
- 유사도 상위 10개 관련 문서의 메타데이터 및 텍스트 원본을 반환, 반환된 데이터와 사용자 질문을 LLM에 요청하여 질문에 대한 내용을 10개 관련 문서 내에서 정리하여 최종 답변 반환

문제 및 해결 과정

- 기간 초기에는 사내 Document Parse를 사용할 수 없는 상황이었으며, PyPDFLoader를 사용하여 문서를 추출했지만, **순서가 섞이는 문제**가 발생하여 **PyMuPDFLoader를 사용**. PyMuPDFLoader는 텍스트 구조 및 레이아웃을
  잘 보존하기에 순서 섞임 문제가 사라져 자연스러운 청킹 진행 가능
- 사내 DP를 사용하여 Json 형태로 추출, 이 데이터를 전처리하여, 헤더, 푸터 삭제 정규식을 통해 필요없는 문자 삭제 후 청킹을 진행하였지만,
  표 형식이 제대로 들어가지 않는 문제, 표 섹션은 마크다운 형식으로 원본 저장 및 임베딩 진행
- LLM에 제공한 청크들 내에서 LLM이 답변을 제공해 주지 않는 경우도 있었지만, 이 문제는 LLM 문제로 보며 RAG Fail이 아닌 LLM Fail로 판단하며 Upstage의 Solar Pro 2의 성능 문제로 OpenAI 모델로 변경했을 때 문제가 없는 것을 확인

느낀 점

- RAG 를 개발하며 생소하지만, 검색했을 때 가이드 라인대로 결과가 나올 때는 재미를 느꼈습니다. 하지만 성능이 생각보다 많이 좋지 않아
  어려움을 느꼈습니다.
- 하이브리드 서치, 청킹 전략, 유사도 점수 평가 등 다양한 방법들이
  있었지만, 시간이 부족하여 다 진행해 보지 않은 게 아쉬웠습니다. 사내 DP 로 추출하는 Json 형태는
  효율적으로 청킹하고 전처리할 수 있는 형태였기에 다양한 것을 해보고 싶었습니다.
- 어떤 문서에도 범용적으로 적용할 수 있는 RAG 파이프라인을 개발하고 싶었지만, 문서의 형태가 다 다르고 나오는 특수문자들도 다르기 때문에 RAG 의 성능을 올리려면 문서 형태에 맞게 전처리, 정규식 등을 따로 적용해야 하므로
  방안을 생각해 내지 못 했습니다. RAG 프로젝트를 진행할 날이 온다면 우선 이런 방법부터 생각해 볼 것 같습니다.

팀 프로젝트

프로젝트명 : Toco(메모 유사도 기반 마인드맵 시각화)

팀원 : 2 명(Front-End 및 Vector 값 기반 클러스터링 1 명, Back-End 1 명)

기간 : [2025. 03~2025.06]

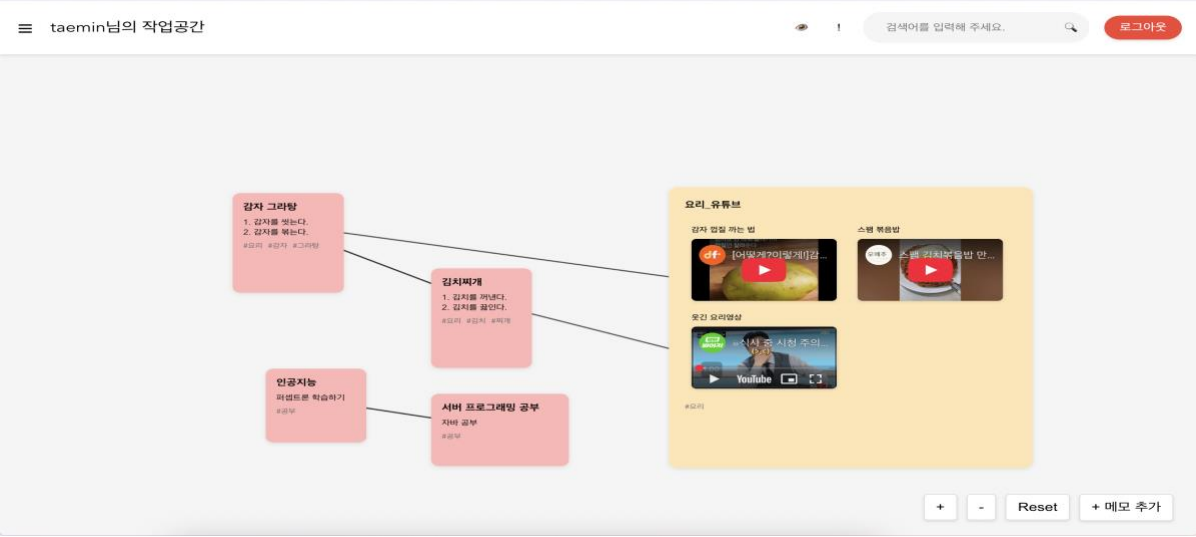
목적 : 사용자가 작성한 메모 및 유튜브 영상들의 카테고리 유사도를 기반으로 메모 마인드맵 형식의 간선으로 연결하여 시각화, 메모 관리 용이

Front-End GitHub : https://github.com/taemin01/PROJECT_TOCO_FRONTEND

Back-End GitHub : https://github.com/wnstjq0915/PROJECT_TOCO_API

사용 기술

Front-End	Vue.js(Composition API), HTML/CSS/JavaScript
Back-End	Python, FastAPI
Database	MySQL, Chroma DB
협업 및 버전관리	Git, GitHub
외부 서비스	OpenAI API



메모 원본 내용 및 유튜브 영상 제목을 통해 OpenAI API 사용하여
카테고리 추출 후 데이터베이스 저장 및 OpenAI Embeddings로
Vector 값 변환 후 Chroma DB(Vector DB)에 저장
Vector 값 기반 Scikit-learn DBSCAN 알고리즘으로 클러스터링 진행
군집화되어 있는 메모들을 Front-End에서 연결하여 간선 연결하여 시각화
유튜브 영상은 iframe 태그에 형식 맞춰준 URL 명시

드롭다운 메뉴를 통해 일반 메모와 유튜브 영상 삽입이 가능한 카테고리 메모로
나누어 메모 작성을 진행, 유튜브 영상은 여러 개 삽입 가능하도록 화면 제작

메인 화면에서 메모 요소들을 더블 클릭하면 메모 유형별로 상세 페이지 개발
일반 메모는 텍스트 수정 및 삭제, 카테고리 메모는 유튜브 영상에 대해
개수 추가, 기존 영상 수정 및 삭제 가능

주요 기능

- 사용자 작성 메모 클러스터링 기반 간선 연결
- 메모 자유 이동 및 편집, 삭제 기능

담당 역할

- Vue.js(Composition API) 화면, 로직, API 연동 전체적인 프론트엔드 담당
- mini PC(home server) 구축, Chroma DB 관리 및 구조 설계 후 DB 구축
- OpenAI API 사용하여 카테고리 추출 및 Embeddings로 벡터값 변환
- 클러스터링 로직 구현(Scikit-learn DBSCAN 알고리즘 사용)

문제 및 해결 과정

- 클러스터링 과정 노이즈 : 밀도가 낮아 노이즈 값이 발생하는 문제를 반복문 통해 **하이퍼 파라미터 조정하여 최적의 값을 찾아 클러스터링 진행**
- 위 방법으로도 성능이 좋지 않아 eps 값을 동적처리로 변경,
  k 최근접 이웃 거리 그래프에서 꺾이는 지점을 자동으로 찾아 eps를 추정하는**knee point 기반 방식으로 eps를 동적으로 지정하여 클러스터링 진행**

보완 및 추가 개발 예정 기능

- 간선 덮임 문제 : 간선의 z-index를 조정하여 메모 div 영역에 가려지지 않게 조정
- 유튜브 영상 카테고리 : 현재 사용자가 작성하는 영상 제목으로 카테고리 추출 중이지만 추후 **영상 자막을 통해 요약하여 카테고리 추출 기능 개발 예정**
- 실시간 협업 및 공유 : Web Socket 기반 실시간으로 반영하여 협업 및 공유 가능

AI 활용 및 검증 과정

- Cursor AI 사용 : 화면 스타일을 적용하는 과정에서 간단한 화면 설계서 통해 생성형 AI에 요청 후 빠르게 레이아웃 형성하여 화면을 개발하였습니다.
- Cursor AI Ask 모드로 질문하여 응답을 받아, Agent 모드에 해당 문제에 대한 요청문을 작성하여 해결 및 코드에 대한 설명을 통해 직접 읽어보며 검증 및 부분 수정하여 프로젝트를 진행하였습니다.
- ChromaDB 구조 설계 및 활용, 데이터 검색 및 저장 과정의 코드를 공식 문서 보며 보다 유연한 활용 부분은 생성형 AI를 통해 학습하였습니다.

느낀 점

- Cursor AI 첫 사용으로 기능을 완벽하게 활용하지 못했다고 생각합니다. Rules추가와 문서 기반 AI 채팅 프롬프트 작성 능력이 필요하다 느꼈습니다.
- 프론트엔드 및 클러스터링 기능을 담당하였으며, 화면 디자인 및 OpenAI 텍스트 생성 API가 마크다운 형식으로 반환하여 프롬프트를 수정하였습니다.
- DBSCAN 알고리즘을 사용해 클러스터링 과정에서 노이즈 값으로 인해 어려움을 느꼈지만, 하이퍼 파라미터를 조정하여 해결했습니다. 앞으로도 AI 관련 프로젝트를 진행하고 싶을 만큼 흥미로운 부분이었습니다.

프로젝트명 : Refresh(중고거래 웹 사이트)

팀원 : 4 명(Front-End 2 명, Back-End 2 명)

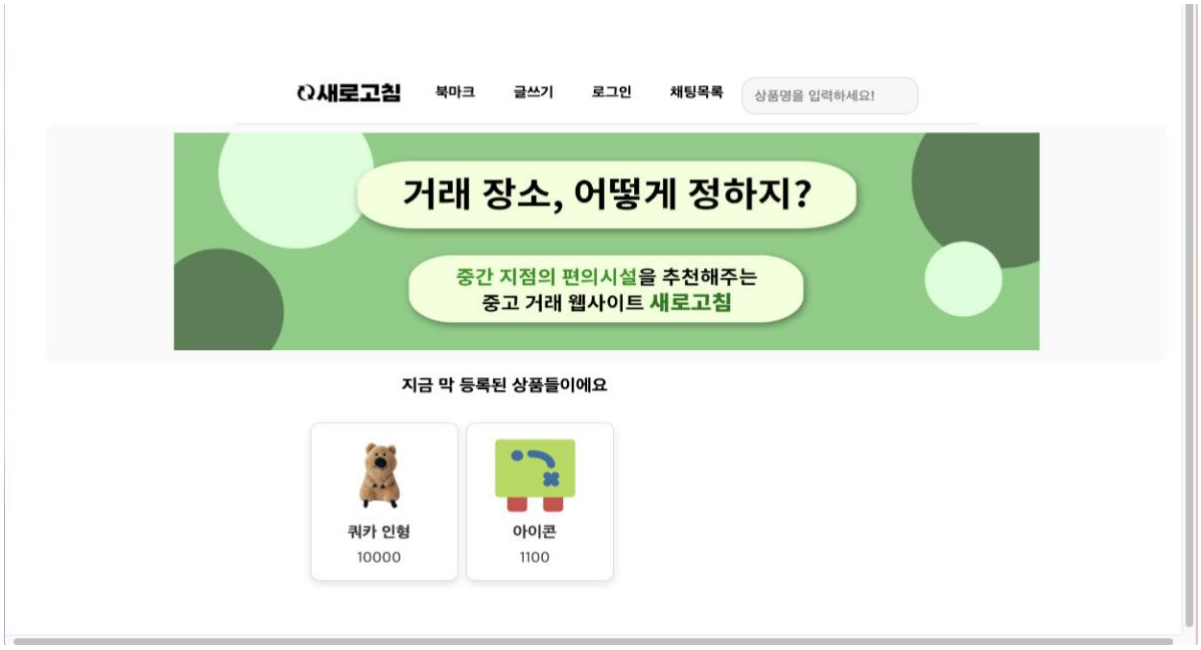
기간 : [2024. 09~2024. 12]

목적 : 거래 장소 설정에 도움을 주기 위해 위도 및 경도 기반으로 중간 거리를 계산하여 추천 위치 제시

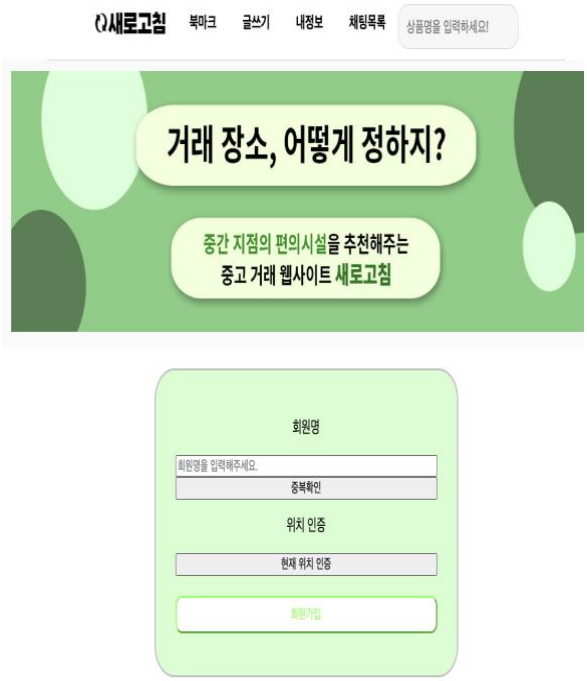
GitHub : https://github.com/taemin01/refresh_market

사용 기술

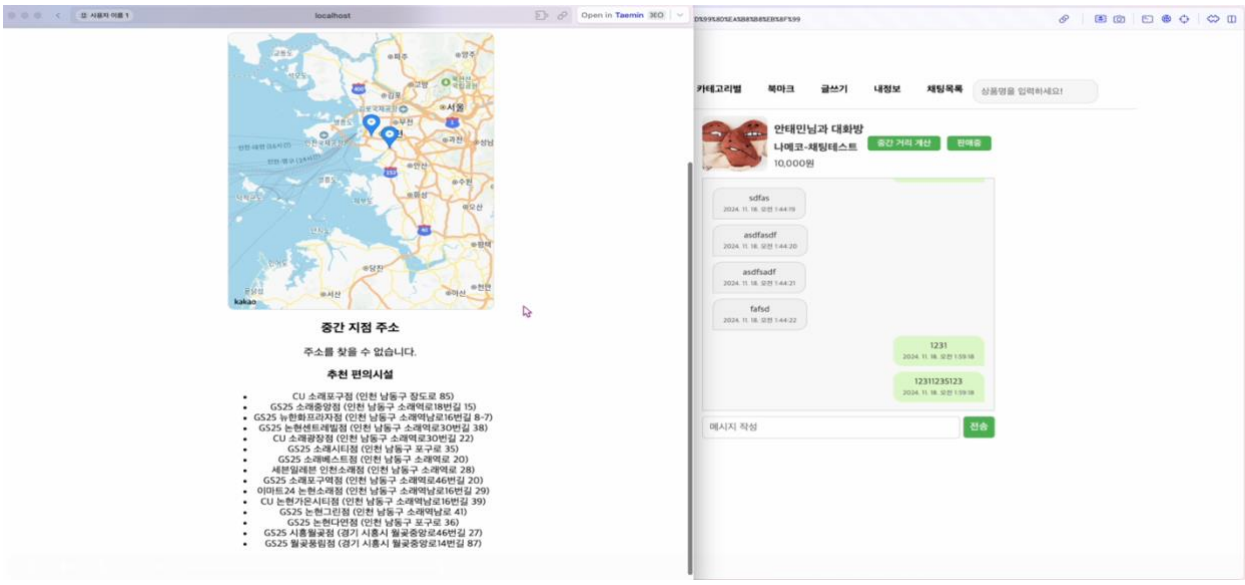
Front-End	React.js, HTML/CSS/JavaScript
Back-End	Java, Spring Boot, JPA
Database	MySQL, Firebase Realtime Database
협업 및 버전관리	Git, GitHub
외부 서비스	Kakao Login API, Kakao Maps API, Geolocation API



프로젝트 메인 화면 배너에 중간 지점의 편의시설 추천해 주는 강점을 강조하여 웹 사이트 설명



메인 화면 로그인 버튼을 통해 카카오 로그인 이후 Kakao Login API 통해 인가 코드 발급 -> 액세스 토큰 발급 -> 카카오 사용자 정보 받아온 후 첫 로그인을 한 사용자일 경우 회원가입 페이지에서 회원명 및 Geolocation API를 사용하여 위치 정보를 받음



회원가입 과정에서 받은 위도 경도를 통해 두 사용자의 중간 좌표를 계산하고, 계산한 중간 지점 위도 경도를 Kakao Maps API Places 라이브러리 category Search에 키워드와 함께 검색하여 추천 편의시설(편의점) 목록을 제시

주요 기능

- 회원가입(카카오 로그인)
- 상품 게시글 CRUD, 게시글 스크랩(좋아요 기능)
- 채팅 및 좌표 기반 중간 거리 계산(중간 지점 근처 편의 시설 추천)

담당 역할

- 카카오 로그인 백엔드 로직, 채팅 기능
- AWS RDS 구축 및 데이터베이스 설계
- 담당 기능 이외의 기능(게시물 검색, 중간 거리 지정 페이지 등) 서포트
- 해당 팀 프로젝트의 팀장 담당(주간 발표 위한 진척도 관리 및 일정 정리)

문제 및 해결 과정

- CORS 문제 : 백엔드 CORS Config 설정으로 출처 허용
- 카카오 메일 비동의 문제 : **HttpOnly 쿠키로 Kakao Id 를 브라우저에 전달**, 회원가입 입력 정보를 보낼 시 쿠키도 포함되어 백엔드로 전송, Kakao Id 와 입력 정보를 데이터베이스에 저장하는 것으로 로직 변경 예정
- 중간 지점 주소 변환 문제 : 직선거리 계산의 한계로 중간 지점(경, 위도)을 **Kakao Map API 의 Places 라이브러리로 주변 편의 시설 주소 목록 출력** 기능 구현

AI 활용 및 검증 과정

- ChatGPT 사용하여, 프로젝트 전반적 시나리오 및 프로젝트 환경 등을 프롬프트로 작성하여 프로젝트 진행, 코드 초안을 받아 RESTful API 코드 작성하였습니다. 백엔드 코드는 Postman 을 통해 테스트를 진행한 후 프론트엔드와 연동하였습니다.
- 프레임워크 및 Kakao API 공식 문서에서 정보를 찾아달라고 하여 필요한 부분에 대한 정보를 ChatGPT 에게 반환받아 프로젝트에 적용하였습니다.

느낀 점

- 처음 사용하는 프레임워크 및 외부 API 들이 익숙하지 않아 어려움을 느꼈지만, 코드를 많이 보며 익숙해지기 위해 노력했습니다.
- RESTful API 라는 것을 접해보며 JSON 데이터 통신을 해보았고, 이 과정에서 지식이 부족하여 데이터 교환 및 프론트엔드 상태 저장 등에 어려움을 느꼈습니다. 백엔드를 담당하였지만, 팀원이 해결하지 못하는 부분을 해결해보며 프론트엔드도 경험해봤습니다. 이 과정이 있었기에 조금 더 빠르게 익숙해졌다고 생각합니다.
- 첫 생성형 AI 를 사용해보며, 프롬프트 작성에 대해 어려움을 느꼈습니다. 제 생각대로 답변이 오지 않았으며, 활용하지 못한다면 생산성이 줄어드는 것이 아닌 배로 늘어날 것이라고 느꼈습니다.
- JWT 토큰, XSS, 쿠키 탈취 등 웹 보안 지식이 다소 어려웠지만, 공부하는 과정에서 이 문제들을 해결하지 않으면 치명적인 문제로 다가올 수 있다는 것을 느꼈습니다.
- 첫 협업 과정에서 Git, GitHub 사용에 어려움을 느꼈습니다. 이슈 관리, 충돌 해결 작업 들이 처음이라 익숙하지 않았으며, 무섭다고 생각했습니다.
  하지만 앞으로의 프로젝트에서는 이슈 관리, 원활한 소통을 통해 협업을 진행해야겠다고 다짐했습니다.
- 첫 데이터베이스 설계를 하게 되었습니다. 잘못된 설계로 인해서인지 데이터베이스 구조를 수정하며 프로젝트를 진행하였습니다. 처음부터 견고하고 안정적인 데이터베이스 설계를 위해 공부가 필요하다 느꼈습니다.

자기소개

안녕하십니까. 빠른 실행력과 강한 적응력 그리고 문제를 끝까지 해결하는 자세를 가진 안태민입니다.

2 학년 2 학기 웹을 주제로 한 팀 프로젝트를 진행하며 웹 개발을 처음 경험했습니다. 당시 CORS, DB 설계, GitHub 사용법에 대한 기초 지식이 부족하여 여러 어려움을 겪었습니다. 하지만 협업을 통해 정해진 기간 내에 모든 문제를 해결하고 프로젝트를 마무리했습니다.

이 경험을 통해 부족한 점을 확인할 수 있었고, JWT 토큰 처리, XSS 등 보안 관련 지식을 보완하며, 안정적이고 유지보수 쉬운 설계를 하는 것이 프로젝트 속도와 성능 향상에 직접적인 영향을 준다는 것을 배웠습니다.

항상 더 좋은 방법은 없을지 고민하고, 협업에 있어서 어떤 기술들을 사용해야 하는지 고민해 왔습니다. 궁금한 부분은 스스로 찾아보며 지식을 꾸준히 채워가고 있습니다.

앞으로도 문제 해결 및 협업 능력을 꾸준히 키워, 가독성 높고 효율적인 코드를 작성하는 개발자로 성장하겠습니다.

이력서를 검토해 주셔서 감사드립니다.
저의 장점과 기술을 활용하여 빠르게 성장하고 팀에 기여할 수 있는 개발자가 되겠습니다. 감사합니다.